Advanced Neural Networks

Prof. Dr. Hani Hagras

Lecture 10: Recent Advances in Neural Networks

Lecture Summary

- *Artificial neural network is still a rapidly developing area, in which there are many open problems.*
- *Contents:*
  - *Neuro-fuzzy networks*
  - *Evolving neural networks*
  - *Spiking Neural Networks*
  - *Reinforcement learning*
- **(NB:** *This lecture will emphasise basic ideas only.***)**

Neuro-fuzzy Networks

Basic idea (*why neuro-fuzzy networks?*)

Truly intelligent systems should be able to make use of all available knowledge, including experimental data, expert or heuristic rules, and physical laws. Neuro-fuzzy networks combine the qualitative reasoning via *fuzzy logic* and the quantitative adaptive data processing via *neural learning*. They allow all the above-mentioned knowledge sources to be incorporated in a single information framework.

Q^2 : *Qualitative fuzzy logic* + *Quantitative neural learning*

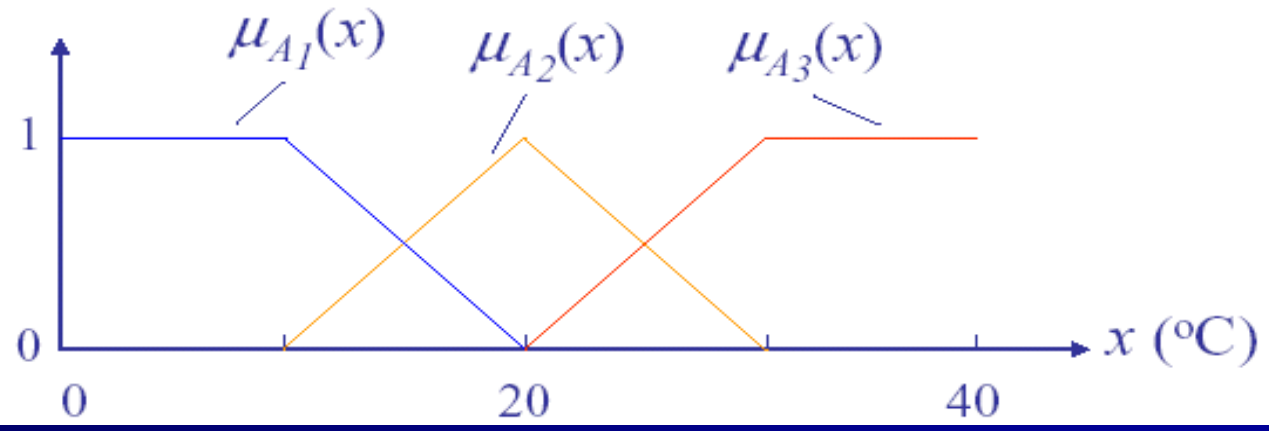
(Interpretability) (Accuracy)

Fuzzy set

A fuzzy set A is specified by a membership function $0 \le \mu_A(x) \le 1$, which gives partial degree of membership of an object x in A.

Example 1: Object x: room temperature.

Three fuzzy sets: A_1: cold, A_2: warm, A_3: hot.



Fuzzy Rule

Fuzzy rule

A fuzzy rule defines the relationship between fuzzy sets (linguistic variables).

Examples:

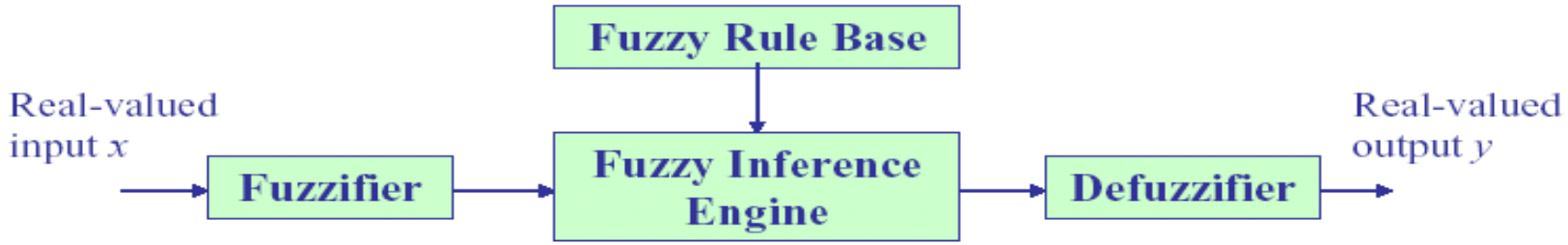
If x is A_1 (room is cold) Then y is B_1 (turn heating up)

If x is A_2 (room is warm) Then y is B_2 (keep heating constant)

If x is A_3 (room is hot) Then y is B_3 (turn heating down)

Fuzzy Inference System

Fuzzy inference (reasoning) system



Fuzzification: real-valued input membership values.

Defuzzification: membership values real-valued output.

Centroid is a popular method for defuzzification:

$$y_i = \frac{\int y \cdot \mu_{B_i}(y)dy}{\int \mu_{B_i}(y)dy} \qquad y = \frac{\sum_i \mu_{A_i}(x) \cdot y_i}{\sum_i \mu_{A_i}(x)}$$

Neuro-fuzzy Network

If we represent the defuzzification as a function $l_k(\mathbf{x})$, the fuzzy rules and the inference system can be represented as follows:

Fuzzy rules: If $\mathbf{x}$ is A_k, then y is $l_k(\mathbf{x})$, $k=1, 2,\ldots, K$.

Inference system:

$$y = \frac{\sum_{k=1}^{K} [\mu_{A_k}(\mathbf{x}) \cdot l_k(\mathbf{x})]}{\sum_{i=1}^{K} \mu_{A_i}(\mathbf{x})} = \sum_{k=1}^{K} [\mu'_{A_k}(\mathbf{x}) \cdot l_k(\mathbf{x})]$$

Neural learning algorithms can be used for constructing $\mu_{A_k}(\mathbf{x})$ and $l_k(\mathbf{x})$, leading to the so-called neuro-fuzzy networks which may provide a good trade-off between model accuracy and interpretability. In pure fuzzy systems, they are determined by expert knowledge.

Evolving Neural Networks

Evolutionary algorithms can be used for evolving neural network architectures, learning rules, connection weight values, and input features.

Neural network representations suitable for evolutionary algorithms are essential here.

New Neuron Models

Is there a better trade-off between the complexity of neurons and the complexity of network connectivity?

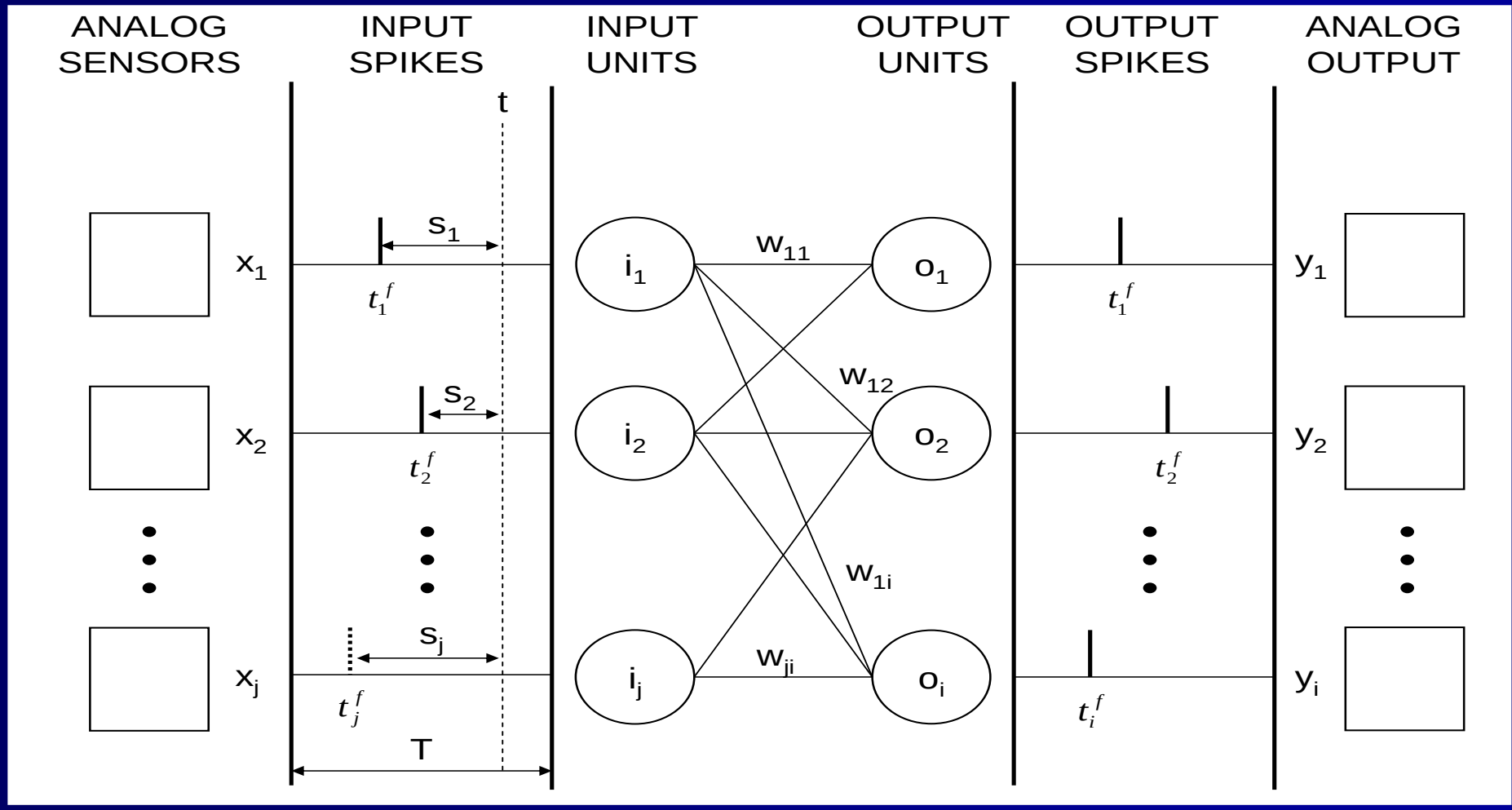
- *Spiking neurons*

The output of a spiking neuron is an impulse train. There are intraneuronal dynamics in spiking neurons, which cannot be described by static input-output mappings. In comparison with static neuron models, spiking neuron models emulate more properties of biological neurons, which have found applications in biologically-inspired control.

Why Spiking Neural Networks

- Most biological neurons communicate by sending pulses across connections to other neurons. Such neurons are called spiking neurons and their networks are termed Spiking Neural Networks.
- SNNs are deemed computationally more powerful than conventional artificial neural network formalisms on the basis of extensive theoretical work by Maass.
- The computational power of SNNs exist because of the intrinsic time-dependent dynamics of spiking neurons that allow the temporal patterns of sensory-motor events to be captured and exploited more efficiently than the other connectionist models (i.e. with fewer neurons and simpler circuits)
- SNNs provide a number of other desirable features such as noise-robustness (tolerance to background noise) and simple real-world interfaces .

A Simple SNN overview



SNN Neuron

- The state of a spiking neuron is described by the voltage difference across its membrane, also known as membrane potential v .
- Incoming spikes can increase or decrease the membrane potential. The neuron emits a spike when the total amount of excitation induced by the incoming excitatory and inhibitory spikes exceeds its firing threshold θ.
- After firing, the membrane potential of the neuron resets its state to a low negative voltage during which it cannot emit a new spike, and it gradually returns to its resting potential. The recharging period is called the *refractory period*.
- There are several models of spiking neurons that account for these properties with various degrees of detail. One of the widely used models for SNN neurons is the Spike Response Model (SRM).

SNN Neuron (Cont)

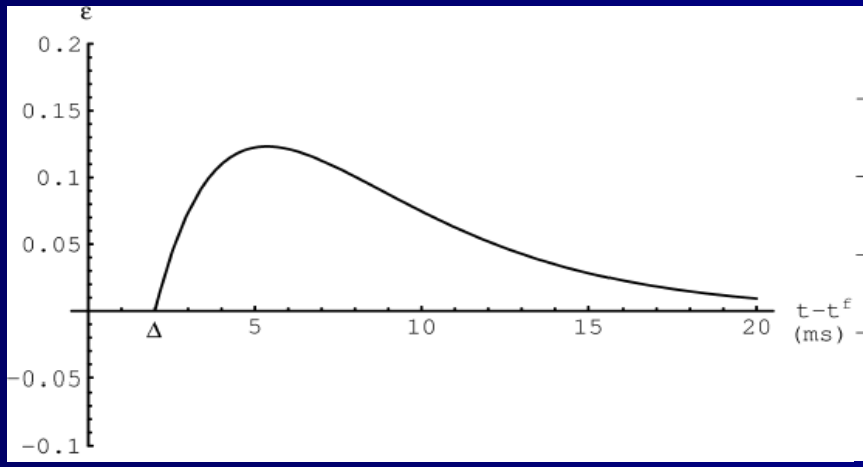
- In the SRM, the effect ε of an incoming spike on the neuron membrane is a function of the difference

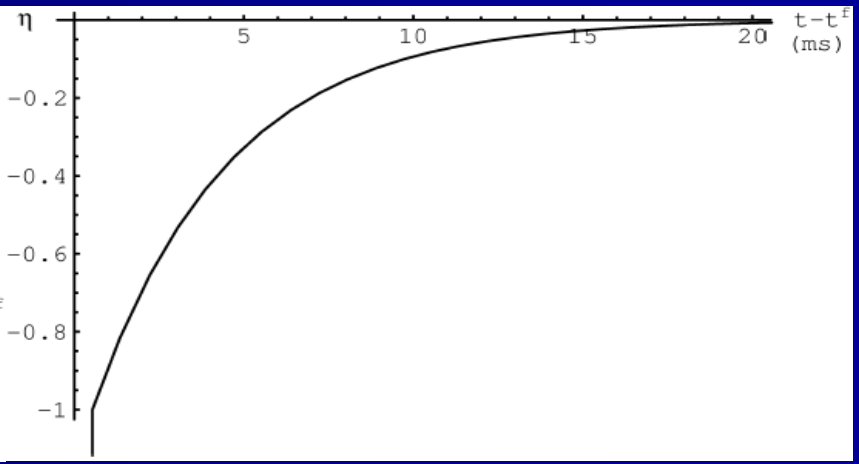
$$s = t - t^f$$

- Where t is the current time and $t\,f$ is the time when the spike was emitted (firing time). The properties of the function are determined by the following:
  - The delay Δ between the generation of a spike at the pre-synaptic neuron and the time of arrival at the synapse.
  - A synaptic time constant τs.
  - A membrane time constant τm.

SNN Neuron (Cont)

- The idea is that a spike emitted by a pre-synaptic neuron takes some time to travel along the axon and once it has reached the synapse, its contribution to the membrane potential is higher as soon as it arrives but gradually fades as time passes.





$$v_i(t_c) = \sum_{j=1}^{N} w_{ij} \sum_{t=1}^{t_c} \varepsilon_j(s_j) + \sum_{t=1}^{t_c} \eta_i(s_i)$$

$$\varepsilon(s) = \exp\left[-(s-\Delta)/\tau_m\right]\left[1-\exp\left(-(s-\Delta)/\tau_s\right)\right] : s \geq \Delta$$
$$0 : s < \Delta$$

$$\eta(s) = -\exp\left[-s/\tau_m\right]$$

Potential of SNNs

- As SNNs have shown to be excellent control systems for biological organisms, they have the potential to produce good control systems for real world applications.
- SNNs can be mapped easily to hardware because the spikes are essentially binary events and the non linear dynamics and the coding of spiking circuits can be provided by spiking times, rather than by non linear, real valued activation functions used in the traditional connectionist neuron models
- In other words, a few logic operations and instructions to move around single bits over time would be sufficient to embed large circuits of spiking neurons that display complex abilities and behaviours into tiny and low power chips.
- Therefore such SNNs will be appropriate control mechanisms for our nano-scale autonomous robots as they can give a very good response dealing with noise using tiny chips that consume little power in the inaccessible environments.

Reinforcement Learning

- *What is reinforcement learning?*

  Reinforcement learning learns an input-output mapping (or policy) through continued interaction with the environment in order to optimise a performance index. It is neither supervised learning nor unsupervised learning.

- *How to represent the input-output mapping in reinforcement learning?*

  In most reinforcement learning applications, both input and output are discrete variables. For example, in behaviour-based robotic control, input variables are perceptual states and outputs are actions. A typical representation for this type of mappings is *lookup table*.

Reinforcement Learning

- *How to interact with the environment?*

  Change the state (*output of the environment and input of the learning system*) by taking an action (*input of the environment and output of the learning system*). The learning system will receive a reward (*a scalar value*).

- *What is the performance index or learning objective?*

  Maximum sum of rewards of a sequence of actions.

Large Language Models

Language Models: Word Embeddings

- One way to encode words: one-hot vectors
  - Want something smarter…

Distributional semantics: account for relationships

- Representations should be close/similar to other words that appear in a similar context

Dense vectors:

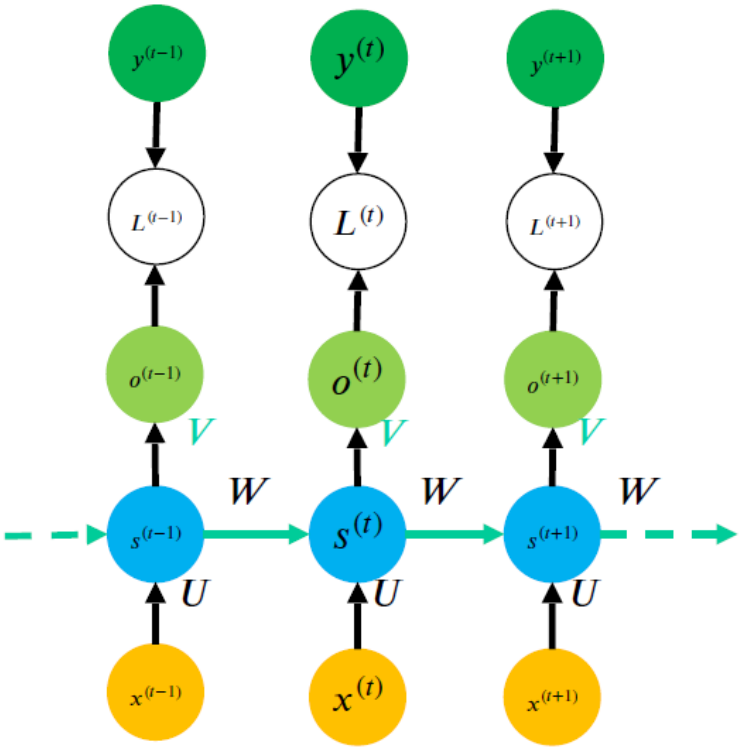
$$\text{dog} = \begin{bmatrix} 0.13 & 0.87 & -0.23 & 0.46 & 0.87 & -0.31 \end{bmatrix}^T$$

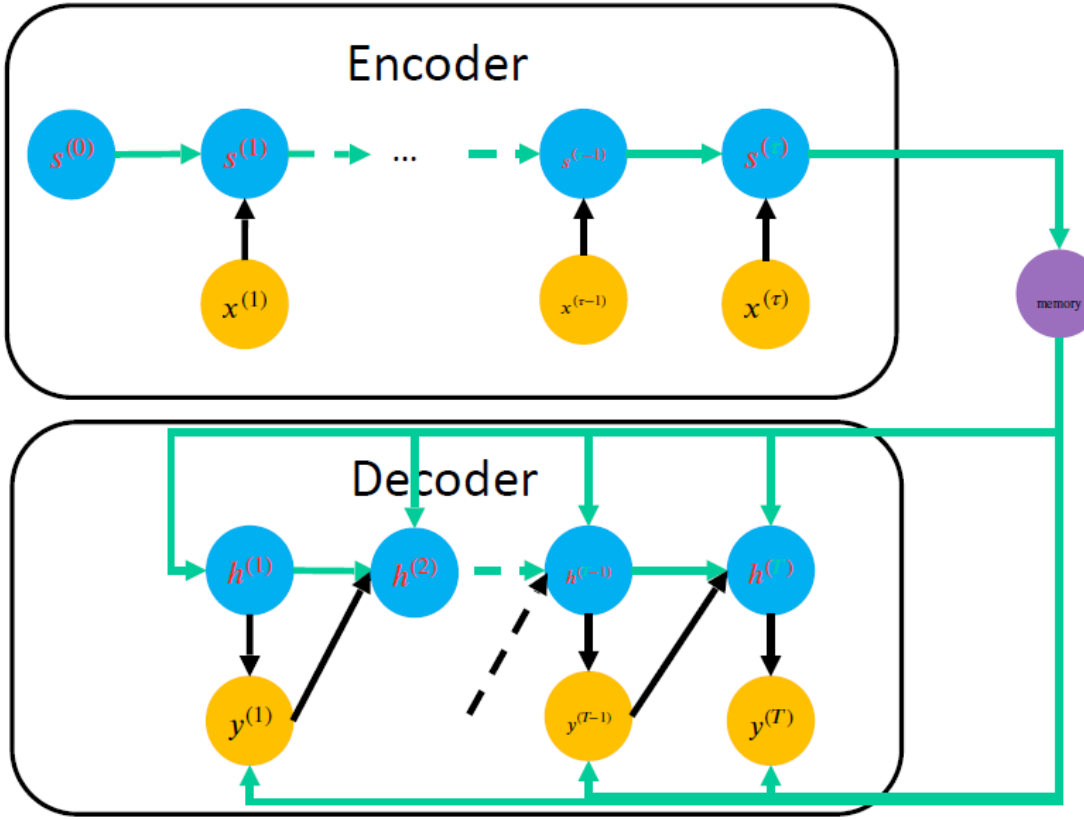
$$\text{cat} = \begin{bmatrix} 0.07 & 1.03 & -0.43 & -0.21 & 1.11 & -0.34 \end{bmatrix}^T$$

AKA **word embeddings**

Language Models: RNN Review

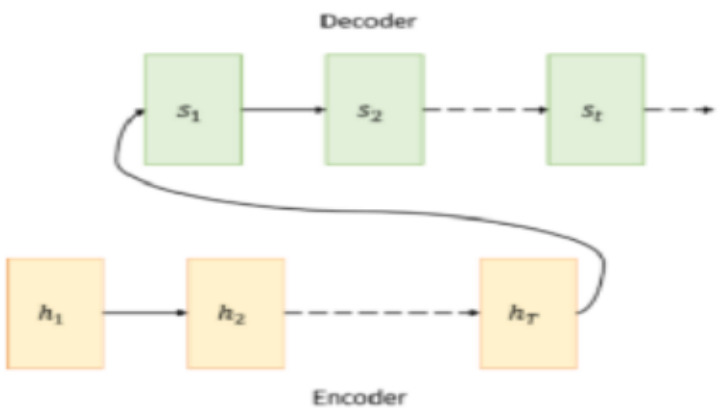
- Classical RNN model / Encoder-Decoder variant:



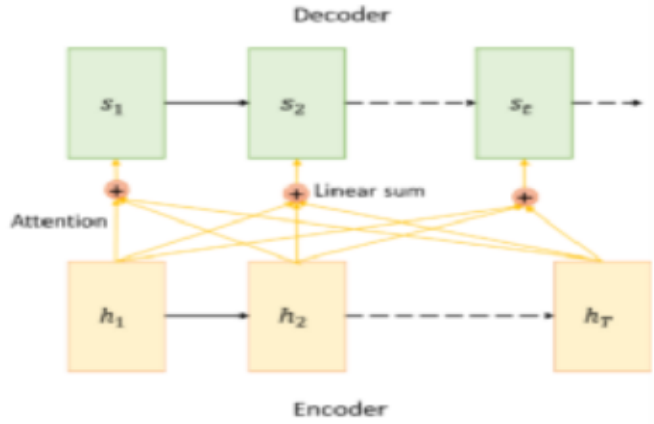


Language Models: Attention

- One challenge: dealing with the hidden state
  - Everything gets compressed there
  - Might lose information
- Solution: **attention** mechanism
  - Similar to residual connections in ResNets!



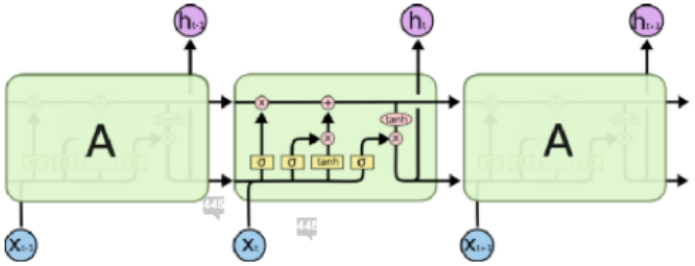
(a) Vanilla Encoder Decoder Architecture



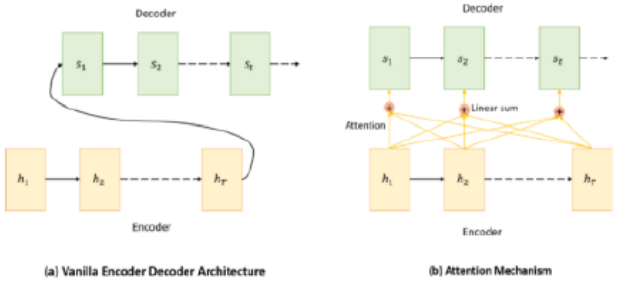
(b) Attention Mechanism

Language Models: Putting it All Together

- Before 2017: best language models
  - Use encoder/decoder architectures based on **RNNs**
  - Use **word embeddings** for word representations
  - Use attention mechanisms



$$\text{dog} = \begin{bmatrix} 0.13 & 0.87 & -0.23 & 0.46 & 0.87 & -0.31 \end{bmatrix}^T$$

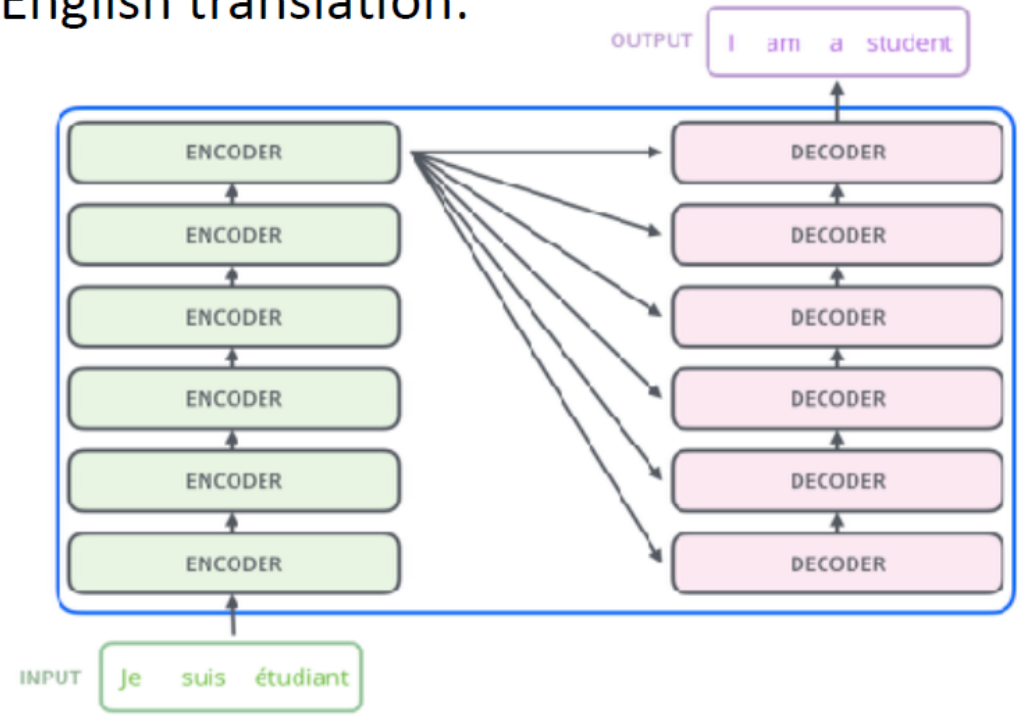


(a) Vanilla Encoder Decoder Architecture

(b) Attention Mechanism

Transformers: Architecture

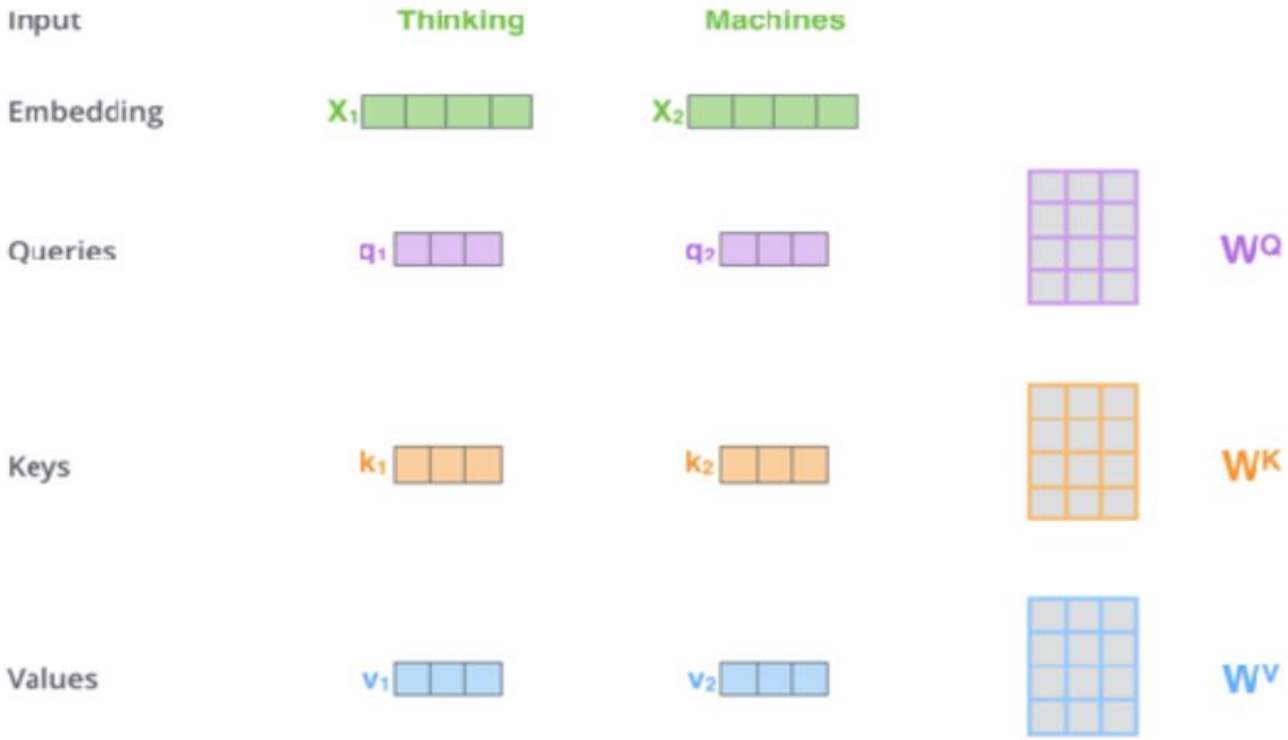
- Sequence-sequence model with **stacked** encoders/decoders:
  - For example, for French-English translation:



Excellent resource: https://jalammar.github.io/illustrated-transformer/

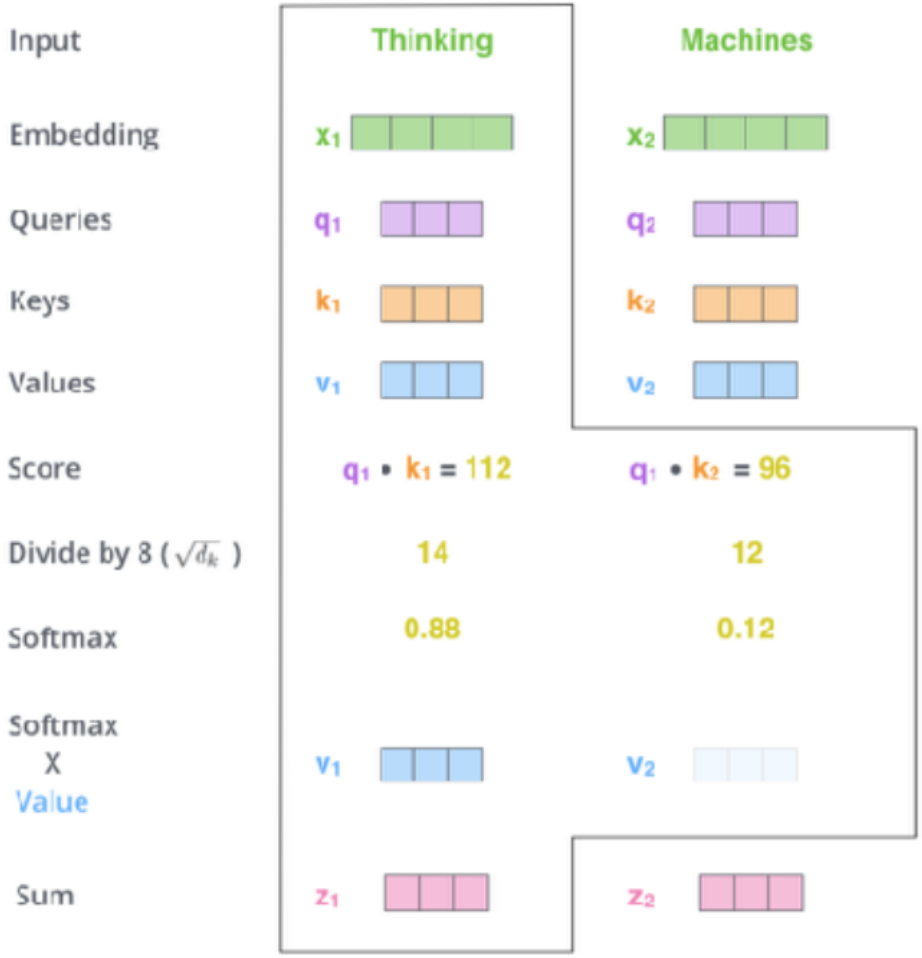
Transformers: Self-Attention

- Self-attention is the key layer in a transformer stack
  - Get 3 vectors for each embedding: **Query**, **Key**, **Value**



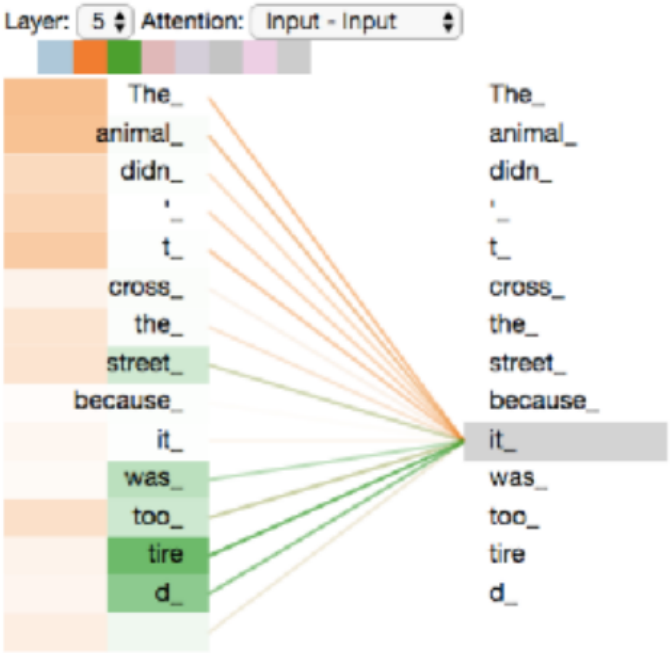
Transformers: Self-Attention

- Self-attention is the key layer in a transformer stack
  - Illustration. Recall the three vectors for each embedding: **Query**, **Key**, **Value**
  - The sum values are the outputs of the self-attention layer
  - Send these to feedforward NNs
- Highly parallelizable!



Transformers: Attention Visualization

- Attention tells us where to focus the information
  - Illustration for a sentence:



How Do Large Language Models (LLMs) Work?

- At their most basic LLMs are statistical pattern-recognition and prediction systems

- LLMs output the next likely word (“token”) in a sentence (“sequence”)
  - token: unit of text e.g. word, character. 1 word ~ 0.75 token
  - sequence: context - section (“window”) of text e.g. sentence, paragraph, book
  - input into chatGPT is 4096 tokens; Claude 2 is 100K tokens

- The likelihood of the next work appearing is determined by
  - the context in which the words are seen in a larger body of text (“corpus”) and
  - the input to the chat

LLMs "Understand" "Meaning"

- Learning from a large corpus allows LLMs to understand the meaning of words.

For example

  - the training data may consist of many sentences beginning with "my favourite colour is…"
  - the next word will be a colour, allowing LLMs to cluster the words "red, blue, green…" into a set that represents the concept of "colour"

- It's important to note that LLMs don't really understand anything. They create statistical patterns that groups similar tokens based on a complex measure of how similar or dissimilar they are.

Data used to Train LLMs

- LLMs are trained in an unsupervised manner on vast quantities of open source and licensed data e.g.
  - The Pile (825GB, incl. web, papers, patents, books, ArXiv, Stack Exchange, maths problems, computer code)
  - Common Crawl (~20B URLs)
  - GPT3: 175B parameters; GPT4: undisclosed: est. 500B – 1000B - bigger is better (for now, at least)
- Responses are refined using question-response pairs ("InstructGPT") from the web, humans or bootstrapped (i.e. the LLM outputs its own pairs)
- reinforcement learning with human feedback (RLHF) is used to reward LLMs to give appropriate responses ("guardrails")
- "Constitutional AI" – trained to filter responses based on e.g. Universal Declaration of Human Rights (Claude 2)

Next Word Prediction

- is influenced by the frequency the word is seen in various contexts
- but there is a degree of randomness so that the word with the highest probability isn't always seen

My favourite colour is

green	9.7%
red	15%
pink	11.6%
puce	2.3%

LLMs can (seem to) be creative

- A consequence of context-based learning and randomness allows the LLMs to generate surprising outputs.
  - Note though that they're not creative in a human sense, but driven by pattern recognition and prediction algorithms

We can use LLMs to:

- identify weakly similar concepts from different disciplines and help understand different disciplines
- generate diverse narratives
- help with ambiguity
- role playing

Beware!

- LLMs may seem to "lie" and "hallucinate" i.e. give what are factually-incorrect responses to questions*
  - as you now know, they're not trained to do give you an objectively correct answer!
- this is some function of training data (e.g. bias), learning, search and probability
- don't believe the outputs – they **always** need checking, at least for now

* LLMs aren't people. They have no intentionality. Don't anthropomorphise them ☺

Interacting with LLMs using "Prompt Engineering"

- Remember that the output of a LLM is determined by both what the system has been trained on and what information you give it
- Prompt engineering means tailoring your questions and input so you can get the most out of an LLM
- Prompts can take many forms, from instructing the LLM to take on a role (e.g. a helpful teacher, a pirate) or guiding the way it should process its output (e.g. "chain of thought" or a particular method).

LLMs can help you engineer prompts

- prompts shouldn't be too precise ("What's the capital of England?"), or too vague ("Tell me about sustainability")

- sometimes you may not know how to ask an LLM to do a task
- ask it what it needs and collaborate with it

E.g.

- "What could I ask you to help me refine my aims for an essay?"
- "Do you need any more information?"

GPT Series of Models

- **GPT**: **G**enerative **P**re-trained **T**ransformer
  - Also built on top of transformer model architecture
  - Essentially the decoder part only
- Goal: generate text (possibly from a **prompt**)
- Scale: huge!
  - GPT-3: 175 billion parameters